实验一 数据库管理

（一）命令操作

1.使用命令启动、关闭 MySQL 服务

启动：net start mysql57

关闭：net stop mysql57

2.登录 MySQL 数据库: mysql -u root -p

3. 查看系统的当前时间： select now()

4. 查看数据库：show databases

5. 创建数据库: create database choose;

6.使用数据库：use dbname

7.删除数据库： drop dbname

8. 存储引擎

show engines;

set default_storage_engine=MyISAM;

9. 创建数据库表

use choose;

set default_storage_engine=InnoDB;

create table my_table(

today datetime,

name char(20)

);

////////////////

alter table my_table engine=MyISAM;

show tables;

desc my_table;

show create table my_table;

（二）练习在 Workbench 上实现上述命令对库与表的操作

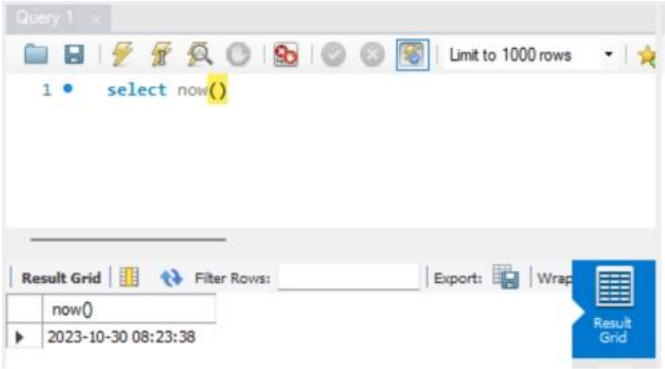
“命令操作”练习作答情况：

题 3：

运行代码：

```
1.  select now();
```

结果截图：

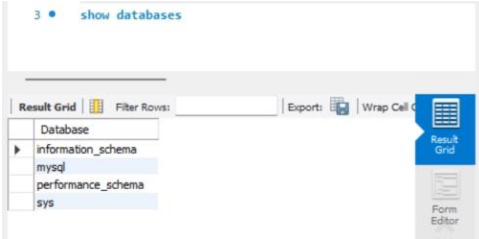


题 4：

运行代码：

```
1.  show databases;
```

结果截图：

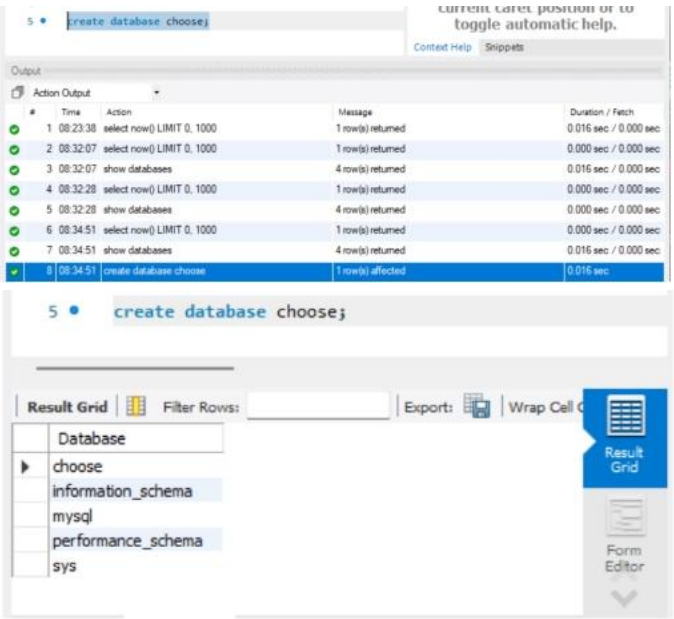


题 5：

运行代码：

```
create database choose;
```

结果截图：

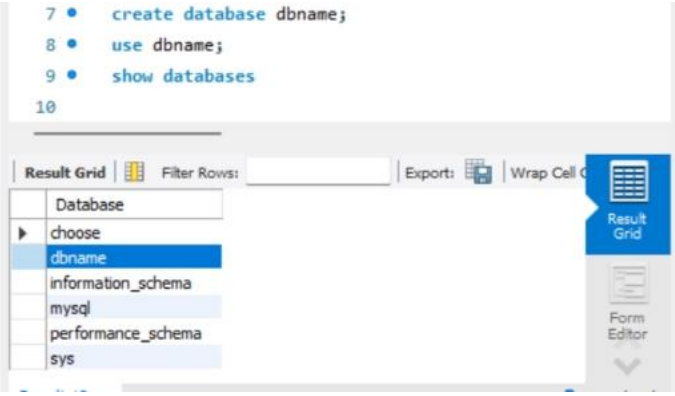


题 6：

运行代码：

```
create database dbname;
use dbname;
show databases;
```

结果截图：

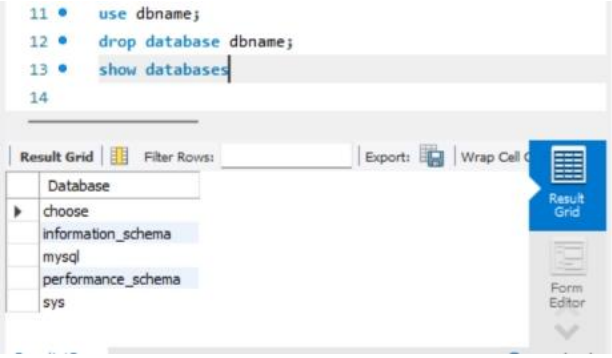


题 7：

运行代码：

```
use dbname;
drop database dbname;
show databases;
```

结果截图：

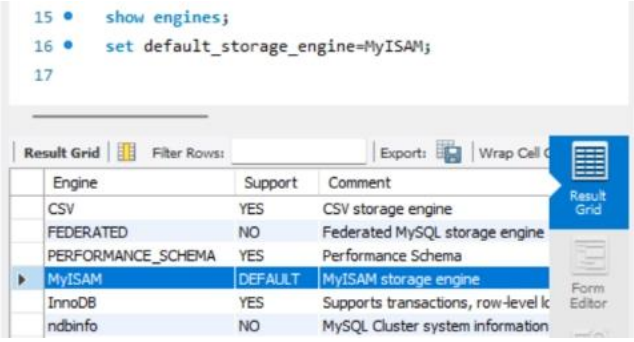


题 8：

运行代码：

```
show engines;
set default_storage_engine=MyISAM;
```

结果截图：

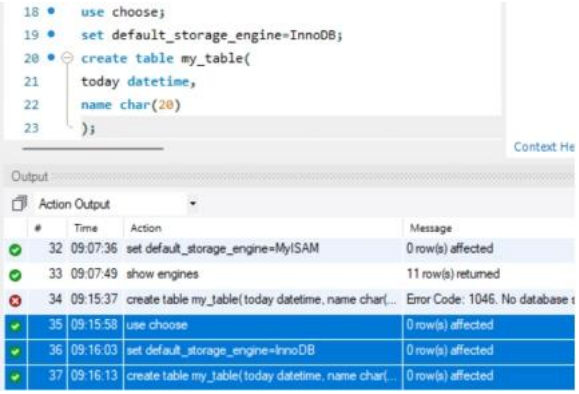


题 9：

① 运行代码：

```
use choose;
set default_storage_engine=InnoDB;
create table my_table(
today datetime,
name char(20)
);
```

结果截图：

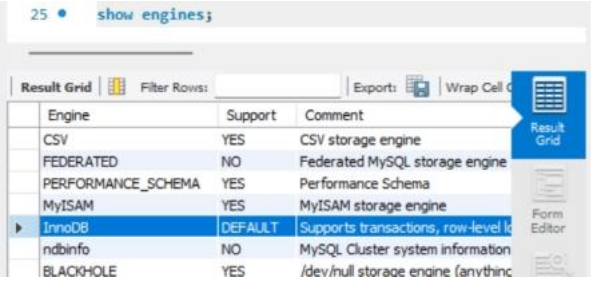


② 展示修改后引擎：

运行代码：

```
show engines;
```

结果截图：



③ 展示数据表：

运行代码：

```
show tables;
```

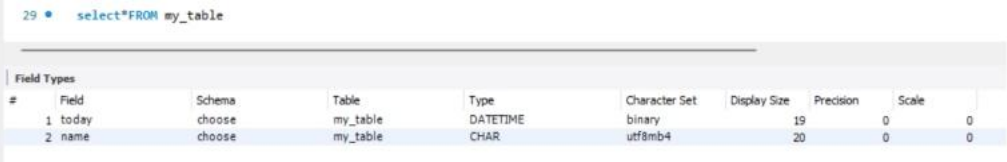
结果截图：

④ 展示数据表中内容：

运行代码：

```
select*FROM my_table;
```

结果截图：

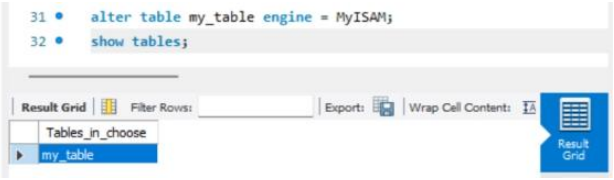


⑤ 展示修改后的数据表

运行代码：

```
alter table my_table engine = MyISAM;
show tables;
```

运行代码及截图：

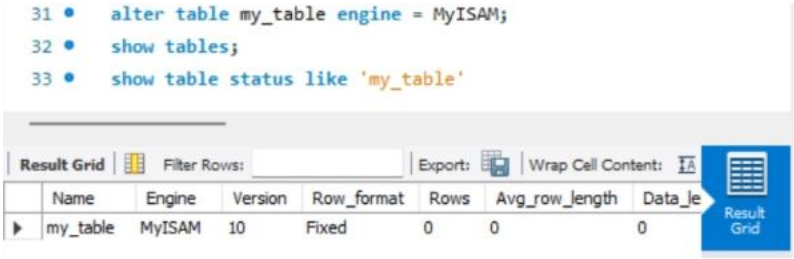


⑥ 展示修改后数据表的引擎：

运行代码：

```
show table status like 'my_table';
```

结果截图：

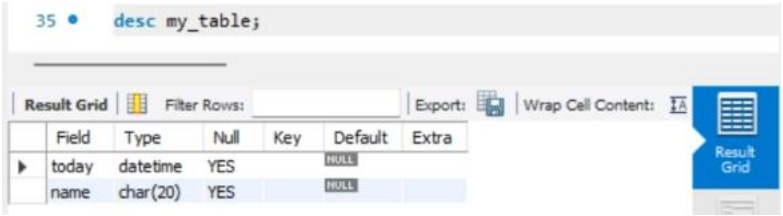


⑦ 查看表的结构：

运行代码：

```
desc my_table;
```

结果截图：



⑧ 查看表的详细信息：

运行代码：

```
show create table my_table;
```

结果截图：

```
37 •  show create table my_table;
```

Form Editor | Navigate:

Table: my_table

Create Table:

```
CREATE TABLE `my_table` (
  `today` datetime DEFAULT NULL,
  `name` char(20) DEFAULT NULL
) ENGINE=MyISAM DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
```

（三）总结与归纳

① mysql 是使用 **C 和“C++”**语言编写的；“C++”擅长面向对象程序设计的同时，还可以进行基于过程的程序设计

② 在 MySQL 中，最常用的方法是使用 **SELECT** 语句查询表数据，还可以通过使用 **WHERE** 子句来筛选数据。WHERE 子句允许在 SELECT 语句中设置条件，只查询符合条件的数据。

SELECT * FROM students;

其中，“*”代表查询所有字段，“FROM students”代表从“students”表中查询数据。如果只想查询表中某些特定的字段，可以使用以下命令：

SELECT field1, field2 FROM students;

其中，“field1”和“field2”代表要查询的字段名。

SELECT * FROM students WHERE age >= 18;

此命令将只返回年龄大于等于18岁的学生数据。

SELECT * FROM students ORDER BY score DESC;

其中，“DESC”表示降序排序，如果要进行升序排序可以使用“ASC”。

SELECT * FROM students LIMIT 10;

此命令将返回“students”表中的前10条数据。

③ 删除数据库或表

```
1.  USE database_name;
```

其中，`database_name` 是包含要删除的表的数据库的名称。

```
DROP DATABASE database_name;
```

其中，`database_name` 是要删除的数据库的名称。

要检查表是否已被删除，可以使用以下代码：

```
SHOW TABLES;
```

该命令将显示当前数据库中的所有表。确保要删除的表不再显示在列表中。

④ `SHOW ENGINES` 显示有关服务器的存储引擎的状态信息。这对于检查是否支持存储引擎或参见默认引擎特别有用。

⑤ 数据表的基本操作

1） 在 MySQL 中，查看表结构可以使用 DESCRIBE 和 SHOW CREATE TABLE 语句。

```
DESCRIBE <表名>;
--  或简写成：
desc <表名>;
```

2） 修改数据表

其语法格式如下：

alter table <表名> [修改选项]

[修改选项] 的语法格式如下：

{ add column <列名> <类型>
| change column <旧列名> <新列名> <新列类型>
| alter column <列名> { set default <默认值> | drop default }
| modify column <列名> <类型>
| drop column <列名>
| rename to <新表名>
| character set <字符集名>
| collate <校对规则名> }

修改表名

mysql 通过 alter table 语句来实现表名的修改，语法规则如下：

```
alter table <旧表名> rename [to] <新表名>;
```

提示：修改表名并不修改表的结构，因此修改名称后的表和修改名称前的表的结构是相同的。

3） 修改/删除字段

3.1 修改字段名称

MySQL 中修改表字段名的语法规则如下：

```
alter table <表名> change <旧字段名> <新字段名> <新数据类型>;
```

3.1 修改字段数据类型

修改字段的数据类型就是把字段的数据类型转换成另一种数据类型。在 mysql 中修改字段数据类型的语法规则如下：

```
alter table <表名> modify <字段名> <数据类型>
```

3.3 删除字段

删除字段是将数据表中的某个字段从表中移除，语法格式如下：

```
alter table <表名> drop <字段名>;
```

4） 数据表添加字段

在开头添加字段

mysql 默认在表的最后位置添加新字段，如果希望在开头位置（第一列的前面）添加新字段，那么可以使用 first 关键字，语法格式如下：

```
alter table <表名> add <新字段名> <数据类型> [约束条件] first;
```

在中间位置添加字段

mysql 除了允许在表的开头位置和末尾位置添加字段外，还允许在中间位置（指定的字段之后）添加字段，此时需要使用 after 关键字，语法格式如下：

```
alter table <表名> add <新字段名> <数据类型> [约束条件] after <已经存在的字段名>;
```

实验二 数据表操作

一、实验目的

(1) 掌握表的基础知识。

(2) 掌握使用数据库管理工具和 SQL 语句创建表的方法。

(3) 掌握表的修改、查看、删除等基本操作方法。

二、实验要求

（1）学生提前准备好实验报告，预习并熟悉实验步骤；

（2）遵守实验室纪律，在规定的时间内完成要求的内容；

（3）撰写并提交实验报告（Word 文档）。要求把每一步操作结果截图。

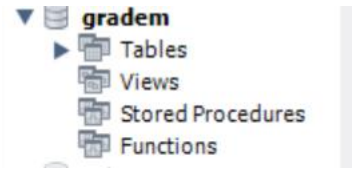
三、实验内容及步骤

创建 gardem 数据库

执行代码：

```
#创建 gradem 数据库
create database gradem;
use gradem;
```

结果截图：



(1)在 gradem 数据库中创建表 1 ~ 表 5 所示结构的表。

表 1　　student 表的表结构

字段名称	数据类型	长度	小数位数	是否允许 NULL 值	说明
sno	char	10		否	主码
sname	varchar	8		是	
ssex	char	2		是	
sbirthday	datetime			是	
saddress	varchar	50		是	
sdept	char	16		是	
speciality	varchar	20		是	

执行代码：

```
#创建 student 表
create table student(
    sno char(10) not null,
```

```
    sname varchar(8),
    ssex char(2),
    sbirthday datetime,
    saddress varchar(50),
    sdept char(16),
    speciality varchar(20),
    primary key(sno)
    );
```

结果截图：

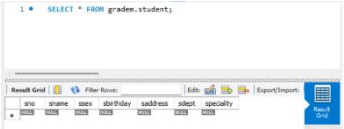


表 2　　course 表(课程名称表)的表结构

字段名称	数据类型	长度	小数位数	是否允许 NULL 值	说明
cno	char	5		否	主码
cname	varchar	20		否	

执行代码：

```
        #创建 course 表
create table course(
    cno char(5) not null,
    cname varchar(20) not null,
    primary key(cno)
);
```

结果截图：

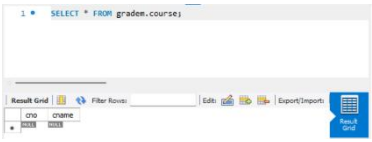


表 3　　sc 表(成绩表)的表结构

字段名称	数据类型	长度	小数位数	是否允许 NULL 值	说明
sno	char	10		否	组合主码、外码
cno	char	5		否	组合主码、外码
degree	decimal	4	1	是	

执行代码：

```
#创建 sc 表
```

```
create table sc(
    sno char(10) not null,
    cno char(5) not null,
    degree decimal(4,1),
    constraint sc_sno foreign key(sno) references student(sno),
    constraint sc_cno foreign key(cno) references course(cno)
);
```

结果截图：

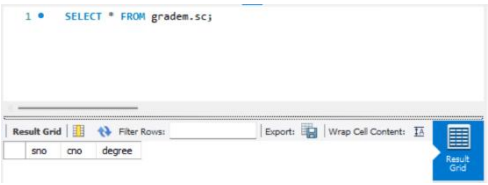


表 4　　teacher 表(教师表)的表结构

字段名称	数据类型	长度	小数位数	是否允许 NULL 值	说明
tno	char	3		否	主码
tname	varchar	8		是	
tsex	char	2		是	
tbirthday	date			是	
tdept	char	16		是	

执行代码：

```
#创建 teacher 表
create table teacher(
    tno char(3) not null primary key,
    tname varchar(8),
    tsex char(2),
    tbirthday date,
    tdept char(16)
);
```

结果截图：

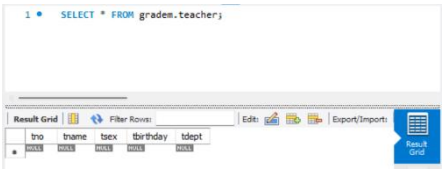


表 5　　teaching 表(授课表)的表结构

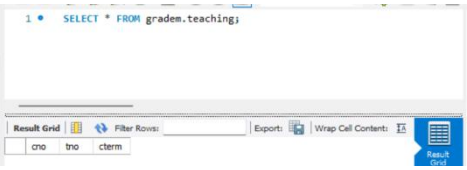
字段名称	数据类型	长度	小数位数	是否允许 NULL 值	说明
cno	char	5		否	组合主码、外码

tno	char	3		否	组合主码、外码
cterm	tinyint	1	0	是	

执行代码：

```
#创建 teaching 表
create table teaching(
    cno char(5) not null,
    tno char(3) not null,
    cterm tinyint(1),
    constraint teaching_cno foreign key(cno) references course(cno),
    constraint teaching_tno foreign key(tno) references teacher(tno)
);
```

结果截图：



(2) 向表 1 至表 5 输入数据记录，见表 6 ~ 表 10。

表 6　　学生关系表 student

sno	sname	ssex	sbirthday	saddress	sdept	speciality
20050101	李勇	男	1987-01-12	山东济南	计算机工程系	计算机应用
20050201	刘晨	女	1988-06-04	山东青岛	信息工程系	电子商务
20050301	王敏	女	1989-12-23	江苏苏州	数学系	数学
20050202	张立	男	1988-08-25	河北唐山	信息工程系	电子商务

执行代码：

```
#向 student 表输入数据
insert into student values
(20050101,'李勇','男','1987-01-12','山东济南','计算机工程系','计算机应用'),
(20050201,'刘晨','女','1988-06-04','山东青岛','信息工程系','电子商务'),
(20050301,'王敏','女','1989-12-23','江苏苏州','数学系','数学'),
(20050202,'张立','男','1988-08-25','河北唐山','信息工程系','电子商务');
select * from student;
```

结果截图：

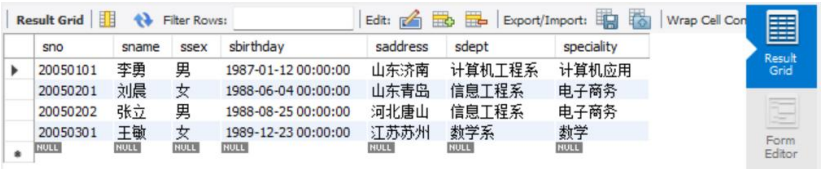


表 7　　课程关系表 course

cno	cname	cno	cname
C01	数据库	C03	信息系统
C02	数学	C04	操作系统

执行代码：

```
#向 course 表输入数据
insert into course values
('C01','数据库'),
('C02','数学'),
('C03','信息系统'),
('C04','操作系统');
select * from course;
```

结果截图：

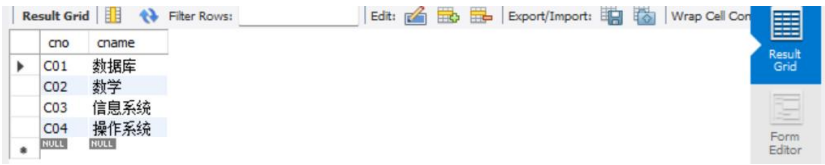


表 8　　成绩表 sc

sno	cno	degree
20050101	C01	92
20050101	C02	85
20050101	C03	88
20050201	C02	90
20050201	C03	80

执行代码：

```
#向 sc 表输入数据
insert into sc values
(20050101,'C01',92),
(20050101,'C02',85),
(20050101,'C03',88),
(20050201,'C02',90),
(20050201,'C03',80);
select * from sc;
```

结果截图：

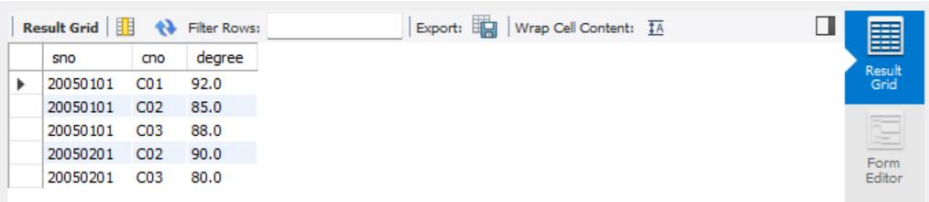


表 9 教师表 teacher

tno	tname	tsex	tbirthday	tdept
101	李新	男	1977-01-12	计算机工程系
102	钱军	女	1968-06-04	计算机工程系
201	王小花	女	1979-12-23	信息工程系
202	张小青	男	1968-08-25	信息工程系

执行代码：

```
#向 teacher 表输入数据
insert into teacher values
(101,'李新','男','1977-01-12','计算机工程系'),
(102,'钱军','女','1968-06-04','计算机工程系'),
(201,'王小花','女','1979-12-23','信息工程系'),
(202,'张小青','男','1968-08-25','信息工程系');
select * from teacher;
```

结果截图：

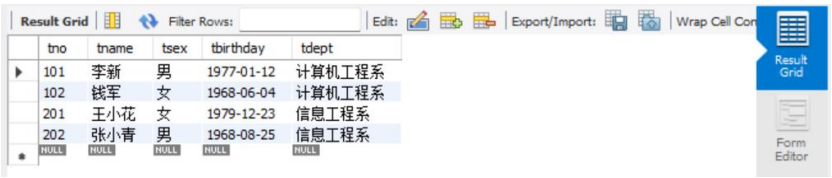


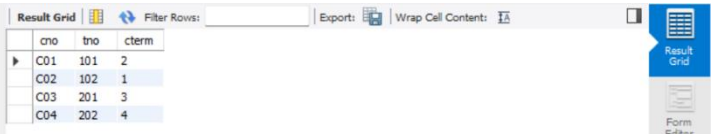
表 10　　授课表 teaching

cno	tno	cterm
C01	101	2
C02	102	1
C03	201	3
C04	202	4

执行代码：

```
#向 teaching 表输入数据
insert into teaching values
('C01',101,2),
('C02',102,1),
('C03',201,3),
('C04',202,4);
select * from teaching;
```

结果截图：



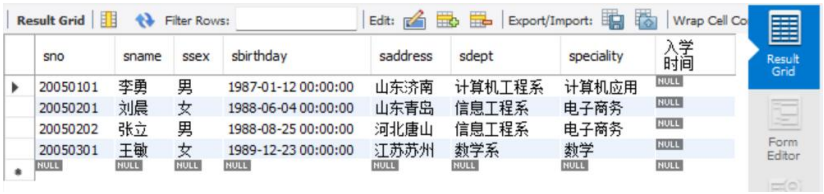
(3) 修改表结构。

① 向 student 表中增加“入学时间”列，其数据类型为日期时间型。

执行代码：

```
alter table student add 入学时间 datetime;
select * from student;
```

结果截图：



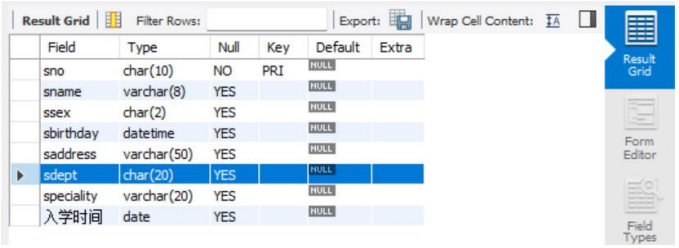
sno	sname	ssex	sbirthday	saddress	sdept	speciality	入学时间
20050101	李勇	男	1987-01-12 00:00:00	山东济南	计算机工程系	计算机应用	NULL
20050201	刘晨	女	1988-06-04 00:00:00	山东青岛	信息工程系	电子商务	NULL
20050202	张立	男	1988-08-25 00:00:00	河北唐山	信息工程系	电子商务	NULL
20050301	王敏	女	1989-12-23 00:00:00	江苏苏州	数学系	数学	NULL

② 将 student 表中的 sdept 字段长度改为 20。

执行代码：

```
alter table student modify sdept char(20);
show columns from student;
```

结果截图：



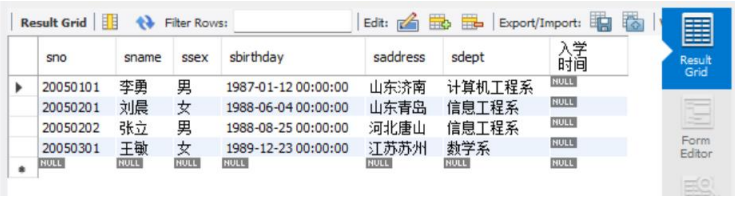
Field	Type	Null	Key	Default	Extra
sno	char(10)	NO	PRI	NULL	
sname	varchar(8)	YES		NULL	
ssex	char(2)	YES		NULL	
sbirthday	datetime	YES		NULL	
saddress	varchar(50)	YES		NULL	
sdept	char(20)	YES		NULL	
speciality	varchar(20)	YES		NULL	
入学时间	date	YES		NULL	

③ 将 student 表中的 speciality 字段删除。

执行代码：

```
alter table student drop speciality;
select * from student;
```

结果截图：



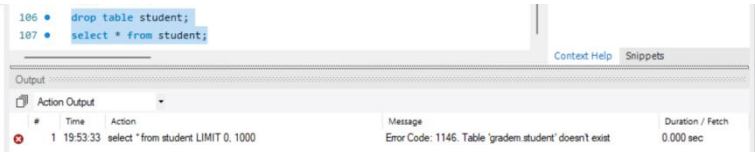
sno	sname	ssex	sbirthday	saddress	sdept	入学时间
20050101	李勇	男	1987-01-12 00:00:00	山东济南	计算机工程系	NULL
20050201	刘晨	女	1988-06-04 00:00:00	山东青岛	信息工程系	NULL
20050202	张立	男	1988-08-25 00:00:00	河北唐山	信息工程系	NULL
20050301	王敏	女	1989-12-23 00:00:00	江苏苏州	数学系	NULL

④ 删除 student 表。

执行代码：

```
drop table student;
select * from student;
```

结果截图：



(4) 利用 SQL 命令（create table、alter table、drop table）完成对表的操作

①利用 create talbe 命令完成 student 表和 course 表的定义。

②利用 drop table 命令实现（3）中的④。

【已按要求完成在每个小题后面】

实验三 数据查询 1（单表查询）

一、实验目的

- 熟练掌握 SELECT 简单查询语句
- 理解并应用比较运算符、逻辑运算符
- 理解并应用聚合函数与 GROUP BY 子句

二、实验要求

（1）学生提前准备好实验报告，预习并熟悉实验步骤；

（2）遵守实验室纪律，在规定的时间内完成要求的内容；

（3）撰写并提交实验报告（Word 文档）。要求把每一步操作结果截图。

三、实验内容及步骤

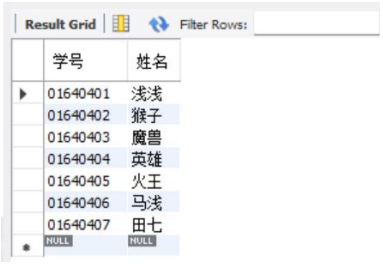
实验数据见学习通材料中的《实验数据创建》。

1 查询学生信息表 student 中的学号、姓名

执行代码：

```
select 学号,姓名 from student;
```

结果截图：



Result Grid　Filter Rows:

学号	姓名
01640401	浅浅
01640402	猴子
01640403	魔兽
01640404	英雄
01640405	火王
01640406	马浅
01640407	田七
NULL	NULL

2 查询学生信息表 student 中的学号、姓名、年龄

执行代码：

```
select 学号,姓名,year(now())-year(出生日期) as 年龄 from student;
```

结果截图：

学号	姓名	年龄
01640401	浅浅	8
01640402	猴子	23
01640403	魔兽	23
01640404	英雄	25
01640405	火王	31
01640406	马浅	22
01640407	田七	21

3 查询图书信息表 book 的折后价（优惠 25%），折后价=单价*0.75

执行代码：

```
select 单价*0.75 as 折后价 from book;
```

结果截图：

折后价
13.5000
31.5000
11.2500
37.5000
5.1000
7.8750
21.0000
7.8750

4 查询课程信息表 course 中的学分少于 4 的课程信息

执行代码：

```
select * from course where 学分<4;
```

结果截图：

课程号	课程名称	学分	学院编号
1	Java语言程序设计	3	1
2	MySQL数据库	2	2
4	C++	2	2
5	系统管理	2	5
NULL	NULL	NULL	NULL

6 在学生信息表 student 中查询“01”班的“女”生有哪些

执行代码：

```
select * from student where 班级编号='01' and 性别='女';
```

结果截图：

学号	姓名	性别	出生日期	班级编号
01640401	浅浅	女	2015-05-06 00:00:00	01
NULL	NULL	NULL	NULL	NULL

7 在课程信息表 course 中查询学分在[2,4]范围内的课程信息。

执行代码：

```
select * from course where 2<=学分<=4;
```

结果截图：

课程号	课程名称	学分	学院编号
1	Java语言程序设计	3	1
2	MySQL数据库	2	2
3	C语言程序设计	4	1
4	C++	2	2
5	系统管理	2	5
6	高等数学	5	1
NULL	NULL	NULL	NULL

8 在学生信息表 student 中查询“01”班、“03”班的学生详细信息。

执行代码：

```
select * from student where 班级编号 in('01','03');
```

结果截图：

学号	姓名	性别	出生日期	班级编号
01640401	浅浅	女	2015-05-06 00:00:00	01
01640403	魔兽	男	2000-02-20 00:00:00	01
01640405	火王	男	1992-04-01 00:00:00	03
NULL	NULL	NULL	NULL	NULL

10 查询课程信息表中课程名称包含“数据库”字符的课程名称和学分。

执行代码：

```
select 课程名称,学分 from course where 课程名称 like '%数据库%';
```

结果截图：

课程名称	学分
MySQL数据库	2

11 查询学生信息表中姓名的第二字是“浅”的学生信息。

执行代码：

```
select * from student where 姓名 like '_浅%';
```

结果截图：

学号	姓名	性别	出生日期	班级编号
01640401	浅浅	女	2015-05-06 00:00:00	01
01640406	马浅	女	2001-03-04 00:00:00	04
NULL	NULL	马浅	NULL	NULL

12 查询学生选课表中成绩为空的信息

执行代码：

```
select * from choose where 成绩 is null;
```

结果截图：

学号	课程号	成绩	选课时间
NULL	NULL	NULL	NULL

13 查询学生选课表 choose 选修"1"号课程的成绩，并按成绩降序排列，

执行代码：

```
select 成绩 from choose where 课程号='1' order by 成绩 desc;
```

结果截图：

成绩
80
50
0

14 查询学生信息表 student 的学生的详细信息，按年龄升序排列。

执行代码：

```
select * from student order by year(now())-year(出生日期);
```

结果截图：

学号	姓名	性别	出生日期	班级编号
01640401	浅浅	女	2015-05-06 00:00:00	01
01640407	田七	男	2002-03-04 00:00:00	02
01640406	马浅	女	2001-03-04 00:00:00	04
01640402	猴子	男	2000-02-20 00:00:00	02
01640403	魔兽	男	2000-02-20 00:00:00	01
01640404	英雄	女	1998-03-18 00:00:00	02
01640405	火王	男	1992-04-01 00:00:00	03
NULL	NULL	NULL	NULL	NULL

15 查询学生选课表中选修“1”号课程的前三名的成绩

执行代码：

```
select 成绩 from choose where 课程号='1' order by 成绩 desc limit 3;
```

结果截图：

成绩
80
50
0

16 查询选了课的学生学号

执行代码：

```
select distinct 学号 from choose;
```

结果截图：

学号
01640401
01640403
01640405
01640402
01640404

17 查询学生信息表 student 中学生人数

执行代码：

```
select count(*) as 学生人数 from student;
```

结果截图：

学生人数
7

18 查询学生选课表 choose 中的“1”号课程成绩的最高分和最低分。

执行代码：

```
select max(成绩) as 最高分,min(成绩) as 最低分 from choose
where 课程号='1';
```

结果截图：

	最高分	最低分
▶	80	0

19 查询订单表 dingdan 中总的下单数量和平均下单数量。

执行代码：

```
select sum(下单数量) as 总下单量,avg(下单数量) as 平均下单量
from dingdan;
```

结果截图：

	总下单量	平均下单量
▶	24	3.4286

20 查询订单表 dingdan 中“2”号买家的总的下单数量和平均下单数量。

执行代码：

```
select sum(下单数量) as 总下单量,avg(下单数量) as 平均下单量
from dingdan where 买家id='buyer2';
```

结果截图：

	总下单量	平均下单量
▶	2	2.0000

21 将学生信息表 student 按照“班级编号”分组，统计各班级的学生人数。

执行代码：

```
select 班级编号,count(*) as 班级人数 from student group by 班级编号;
```

结果截图：

	班级编号	班级人数
▶	01	2
	02	3
	03	1
	04	1

22 查询订单表 dingdan 中每位买家的总的下单数量和平均下单数量。

执行代码：

```
select 买家id,sum(下单数量),avg(下单数量)
from dingdan group by 买家id;
```

结果截图：

	买家id	sum(下单数量)	avg(下单数量)
▶	buyer1	5	5.0000
	buyer2	2	2.0000
	buyer3	6	6.0000
	buyer4	1	1.0000
	buyer5	4	4.0000
	buyer6	2	2.0000
	buyer7	4	4.0000

23 统计每个学生选修了多少门课程，以及该学生所获得的总成绩和平均成绩。

执行代码：

```
select count(学号) as 选修课程门数,sum(成绩) as 总成绩,
avg(成绩) as 平均成绩
from choose group by 学号;
```

结果截图：

选修课程门数	总成绩	平均成绩
3	150	50.0000
1	70	70.0000
2	170	85.0000
1	0	0.0000
1	0	0.0000

24 统计选课门数超过 2 门的学号和课程门数。

执行代码：

```
select 学号,count(学号) as 课程门数
from choose group by 学号 having count(学号)>2;
```

结果截图：

学号	课程门数
01640401	3

实验四 数据查询 2（多表查询）

一、实验目的

- 熟练掌握 SELECT 简单查询语句
- 理解并应用比较运算符、逻辑运算符
- 理解并应用聚合函数与 GROUP BY 子句

二、实验要求

（1）学生提前准备好实验报告，预习并熟悉实验步骤；

（2）遵守实验室纪律，在规定的时间内完成要求的内容；

（3）撰写并提交实验报告（Word 文档）。要求把每一步操作结果截图。

三、实验内容及步骤

实验数据见学习通材料中的《实验数据创建》。

1. 基于图书销售管理数据库，查询 4 号订单买家的详细信息。

执行代码：

```
select * from dingdan;
select * from buyer where 买家id in
(select 买家id from dingdan where 订单id like '%4%');
```

结果截图：

买家id	姓名	性别	家庭住址	联系电话
buyer4	孙丹	女	武生院学东八102	18295621876
NULL	NULL	NULL	NULL	NULL

2. 基于学生选课数据库，查询学生的详细信息。

执行代码：

```
select * from student join classes
on student.班级编号 = classes.班级编号;
```

结果截图：

Result Grid　Filter Rows:　Export:　Wrap Cell Content:

学号	姓名	性别	出生日期	班级编号	班级编号	班级名称	学院编号
01640401	浅浅	女	2015-05-06 00:00:00	01	01	19计算机科学与技术1班	3
01640402	猴子	男	2000-02-20 00:00:00	02	02	19计算机科学与技术2班	3
01640403	魔兽	男	2000-02-20 00:00:00	01	01	19计算机科学与技术1班	3
01640404	英雄	女	1998-03-18 00:00:00	02	02	19计算机科学与技术2班	3
01640405	火王	男	1992-04-01 00:00:00	03	03	19计算机科学与技术3班	3
01640406	马浅	女	2001-03-04 00:00:00	04	04	19信息管理1班	4
01640407	田七	男	2002-03-04 00:00:00	02	02	19计算机科学与技术2班	3

3. 基于学生选课数据库，查询班级人数少于 2 人的班级信息。

执行代码：

```
select 班级名称,count(*) as 班级人数
from student join classes on student.班级编号 = classes.班级编号
group by student.班级编号 having count(*)<2;
```

结果截图：

Result Grid　Filter Rows:

	班级名称	班级人数
▶	19计算机科学与技术3班	1
	19信息管理1班	1

4. 基于学生选课数据库，查询每个学生的选课门数。

执行代码：

```
select 姓名,count(成绩) as 选课门数 from student left join choose
on student.学号 = choose.学号 group by(student.学号);
```

结果截图：

	姓名	选课门数
▶	浅浅	3
	猴子	1
	魔兽	2
	英雄	1
	火王	1
	马浅	0
	田七	0

5. 基于学生信息表 student 和班级信息表 classes 查询每个班级的男生人数，

执行代码：

```
select 班级名称,count(*) as 男生人数 from student right join classes
on student.班级编号=classes.班级编号 and 性别='男' group by 班级名称;
```

结果截图：

	班级名称	男生人数
▶	19信息管理1班	1
	19机电1班	1
	19机电2班	1
	19生工1班	1
	19计算机科学与技术1班	1
	19计算机科学与技术2班	2
	19计算机科学与技术3班	1

6. 查询学生信息表 student 的出生日期小于“马浅”出生日期的学生学号、姓名、出生日期。

执行代码：

```
select a.学号,a.姓名,a.出生日期 from student a join student b
on a.出生日期<b.出生日期 where b.姓名='马浅';
```

结果截图：

	学号	姓名	出生日期
▶	01640402	猴子	2000-02-20 00:00:00
	01640403	魔兽	2000-02-20 00:00:00
	01640404	英雄	1998-03-18 00:00:00
	01640405	火王	1992-04-01 00:00:00

7. 查询图书销售管理数据库中买家“彭万里”购买的图书总数量。

执行代码：

```
select 姓名,下单数量 from buyer join dingdan
on buyer.买家id=dingdan.买家id
where buyer.姓名='彭万里';
```

结果截图：

	姓名	下单数量
▶	彭万里	5

8. 查询没有被任何学生选修的课程信息。

方法一：

执行代码：

```
select * from course left join choose on course.课程号=choose.课程号
where choose.学号 is null;
```

结果截图：

	课程号	课程名称	学分	学院编号	学号	课程号	成绩	选课时间
▶	4	C++	2	2	NULL	NULL	NULL	NULL
	5	系统管理	2	5	NULL	NULL	NULL	NULL
	6	高等数学	5	1	NULL	NULL	NULL	NULL

方法二：

执行代码：

```
select * from course where not exists(
select 课程号 from choose where choose.课程号=course.课程号);
```

结果截图：

	课程号	课程名称	学分	学院编号
▶	4	C++	2	2
	5	系统管理	2	5
	6	高等数学	5	1
*	NULL	NULL	NULL	NULL

9. 使用">=ALL"的子查询查询图书信息表 book 中单价最贵的图书信息。

执行代码：

```
select * from book where 单价>=all(select 单价 from book);
```

结果截图：

图书编号	图书名称	作者	单价	出版商
a2018004	时间简史	史蒂·芬霍金	50.00	科学技术出版社
NULL	NULL	NULL	NULL	NULL

10. 复制 student 表结构为学生表，将 student 表中“女”生的学生信息添加到学生表中。

执行代码：

```
create table 学生 like student;
insert into 学生 select * from student where 性别='女';
select * from 学生;
```

结果截图：

学号	姓名	性别	出生日期	班级编号
01640401	浅浅	女	2015-05-06 00:00:00	01
01640404	英雄	女	1998-03-18 00:00:00	02
01640406	马浅	女	2001-03-04 00:00:00	04
NULL	NULL	NULL	NULL	NULL

11. 删除学生“英雄”选修"C++"课程的记录。

执行代码：

```
#删除记录
delete from choose where 学号=(
select 学号 from student where 姓名='英雄')
and 课程号=(
select 课程号 from course where 课程名称='c++');

#查看删除后记录
select choose.学号,choose.课程号,course.课程名称,student.姓名 from choose
join student on choose.学号=student.学号
join course on course.课程号=choose.课程号
where student.姓名='英雄';
```

结果截图：

Result Grid　Filter Rows:

学号	课程号	课程名称	姓名
01640404	3	C语言程序设计	英雄

12. 查询课程信息表 course 课程号为“1”或“2”的课程信息，UNION 连接语句如下。

执行代码：

```
select * from course where 课程号=1
union select * from course where 课程号=2;
```

结果截图：

Result Grid　Filter Rows:　Expo

课程号	课程名称	学分	学院编号
1	Java语言程序设计	3	1
2	MySQL数据库	2	2

实验五 索引和视图

实验 5.1 MySQL 索引

(一) 实验目的：理解索引的功能，了解索引的分类，掌握索引基本操作的语法格式。

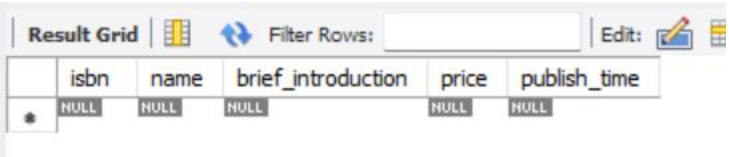
（二）实验内容

1、使用 SQL 语句为表 book2（isbn(出版编号),name(书名),brief_introduction(简介),price(单价),publish_time(出版日期)）创建一个简单索引。存储引擎为 MyISAM、字符集为 GBK。

执行代码：

```
create table book2
(isbn char(20) primary key,
name char(100) not null,
brief_introduction text not null,
price decimal(6,2),
publish_time date not null,
unique index isbn_unique(isbn),
index name_index(name)
)engine=myisam charset=gbk;
select * from book2;
show index from book2;
```

结果截图：



isbn	name	brief_introduction	price	publish_time
NULL	NULL	NULL	NULL	NULL

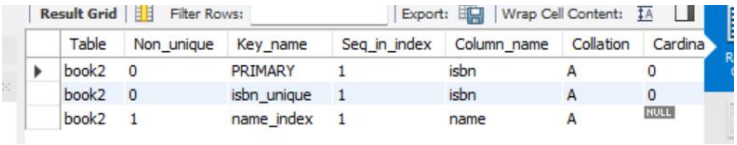


Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardina
book2	0	PRIMARY	1	isbn	A	0
book2	0	isbn_unique	1	isbn	A	0
book2	1	name_index	1	name	A	NULL

2、将 book2 表的图书简介 brief_introduction 字段添加全文索引。

执行代码：

```
alter table book2 add fulltext
index brief_introduction_index(brief_introduction);
show index from book2;
```

结果截图：

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
book2	0	PRIMARY	1	isbn	A	0	NULL	NULL		BTREE			YES	NULL
book2	0	isbn_unique	1	isbn	A	0	NULL	NULL		BTREE			YES	NULL
book2	1	name_index	1	name	A	NULL	NULL	NULL		BTREE			YES	NULL
book2	1	brief_introduction_index	1	brief_introduction	NULL	NULL	NULL	NULL		FULLTEXT			YES	NULL

3、删除书籍 book2 表的复合索引 complex_index。

①在 book2 上创建复合索引

执行代码：

```
#添加复合索引
create index complex_index on book2(isbn,name);
show index from book2;
```

结果截图：

34 20:58:09 create index complex_index on book2(isbn,name) 0 row(s) affected Records: 0 Duplicates: 0 Warning... 0.047 sec

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Coll
book2	0	PRIMARY	1	isbn	A
book2	0	isbn_unique	1	isbn	A
book2	1	name_index	1	name	A
book2	1	complex_index	1	isbn	A
book2	1	complex_index	2	name	A
book2	1	brief_introduction_index	1	brief_introduction	NULL

②删除 book 的复合索引

执行代码：

```
#删除复合索引
drop index complex_index on book2;
show index from book2;
```

结果截图：

36 21:00:04 drop index complex_index on book2 0 row(s) affected Records: 0 Duplicates: 0 Warning... 0.047 sec

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Coll
book2	0	PRIMARY	1	isbn	A
book2	0	isbn_unique	1	isbn	A
book2	1	name_index	1	name	A
book2	1	brief_introduction_index	1	brief_introduction	NULL

Result Grid　Form

（四）总结归纳

1.添加索引

①主键索引：一张表只能有一个主键索引，不允许重复、不允许为 NULL；

```
ALTER TABLE TableName ADD PRIMARY KEY(column_list);
```

②唯一索引：数据列不允许重复，允许为 NULL 值，一张表可有多个唯一索引，索引列的值必须唯一，但允许有空值。如果是组合索引，则列值的组合必须唯一。

```
CREATE UNIQUE INDEX IndexName ON `TableName`(`字段名`(length));
# 或者
ALTER TABLE TableName ADD UNIQUE (column_list);
```

③普通索引：一张表可以创建多个普通索引，一个普通索引可以包含多个字段，允许数据重复，允许 NULL 值插入；

```
CREATE INDEX IndexName ON `TableName`(`字段名`(length));
# 或者
ALTER TABLE TableName ADD INDEX IndexName(`字段名`(length));
```

④全文索引：它查找的是文本中的关键词，主要用于全文检索。（篇幅较长，下文有独立主题说明）

```
alter table info2 add fulltext index addr_index(address);
```

⑤组合索引

```
-- 创建普通组合索引
CREATE INDEX index_name ON table_name(col_name_1,col_name_2);

-- 创建唯一组合索引
CREATE UNIQUE INDEX index_name ON table_name(col_name_1,col_name_2);
```

2.删除索引

```
-- 直接删除索引
DROP INDEX index_name ON table_name;

-- 修改表结构删除索引
ALTER TABLE table_name DROP INDEX index_name;
```

3.修改表结构增加索引

```
ALTER TABLE table_name ADD INDEX index_name(col_name);
```

4.查看索引

```
show index from tablename;
#或者
show keys from tablename;
```

·key 与 index 含义
key 具有两层含义：1.约束（约束和规范数据库的结构完整性）2.索引
index：索引

5.最左前缀原则
在 MySQL 建立联合索引时会遵守最左前缀匹配原则，即最左优先（查询条件精确匹配索引的左边连续一列或几列，则构建对应列的组合索引树），在检索数据时也从联合索引的最左边开始匹配。

实验 5.2. MySQL 视图

（一）实验目的：理解视图的作用，掌握视图的创建、使用、检查、删除等基本操作。

（二）实验内容

5、在 choose 数据库中，为 student 表创建视图 view1。

执行代码：

```
use choose;
create view view1 as select* from student;
select * from view1;
```

结果截图：

Result Grid　Filter Rows:　Export:　Wra

	学号	姓名	性别	出生日期	班级编号
▶	01640401	浅浅	女	2015-05-06 00:00:00	01
	01640402	猴子	男	2000-02-20 00:00:00	02
	01640403	魔兽	男	2000-02-20 00:00:00	01
	01640404	英雄	女	1998-03-18 00:00:00	02
	01640405	火王	男	1992-04-01 00:00:00	03
	01640406	马浅	女	2001-03-04 00:00:00	04
	01640407	田七	男	2002-03-04 00:00:00	02

6、教师要查询某个班学生的各门课程成绩，

执行代码：

```
create view view_成绩 as select classes.班级编号,课程名称,成绩,choose.学号,姓名
from classes,choose,student,course
where classes.班级编号=student.班级编号 and choose.学号=student.学号
and course.课程号=choose.课程号;
select * from view_成绩;
```

结果截图：

	班级编号	课程名称	成绩	学号	姓名
▶	01	Java语言程序设计	50	01640401	浅浅
	01	MySQL数据库	40	01640401	浅浅
	01	C语言程序设计	60	01640401	浅浅
	02	MySQL数据库	70	01640402	猴子
	01	Java语言程序设计	80	01640403	魔兽
	01	MySQL数据库	90	01640403	魔兽
	02	C语言程序设计	0	01640404	英雄
	03	Java语言程序设计	0	01640405	李平

7、统计每一门课程的选课人数

①创建选课视图

执行代码：

```
create view view_选课 as select 学号,课程名称
from course left join choose on course.课程号=choose.课程号;
select * from view_选课;
```

结果截图：

学号	课程名称
01640401	Java语言程序设计
01640403	Java语言程序设计
01640405	Java语言程序设计
01640401	MySQL数据库
01640402	MySQL数据库
01640403	MySQL数据库
01640401	C语言程序设计
01640404	C语言程序设计
NULL	C++
NULL	系统管理
NULL	高等数学

②查询选课人数

执行代码：

```
select 课程名称,count(学号) as 选课人数
from view_选课 group by 课程名称;
```

结果截图：

课程名称	选课人数
Java语言程序设计	3
MySQL数据库	3
C语言程序设计	2
C++	0
系统管理	0
高等数学	0

8、将 view1 视图中学号为“01640405”的学生的姓名改为“李平”。

执行代码;

```
update view1 set 姓名='李平' where 学号='01640405';
select * from view1;
```

结果截图：

8 23:23:21 update view1 set 姓名='李平' where 学号='01640405' 1 row(s) affected Rows matched: 1 Changed: 1 War... 0.016 sec

Result Grid　Filter Rows:　Export:　Wrap Cell Conter

学号	姓名	性别	出生日期	班级编号
01640401	浅浅	女	2015-05-06 00:00:00	01
01640402	猴子	男	2000-02-20 00:00:00	02
01640403	魔兽	男	2000-02-20 00:00:00	01
01640404	英雄	女	1998-03-18 00:00:00	02
01640405	李平	男	1992-04-01 00:00:00	03
01640406	马浅	女	2001-03-04 00:00:00	04
01640407	田七	男	2002-03-04 00:00:00	02

9、删除 view1 视图中学号为“01640407”的记录。

执行代码：

```
delete from view1 where 学号='01640407';
```

结果截图：

10 23:29:11 delete from view1 where 学号='01640407' 1 row(s) affected

学号	姓名	性别	出生日期	班级编号
01640401	浅浅	女	2015-05-06 00:00:00	01
01640402	猴子	男	2000-02-20 00:00:00	02
01640403	魔兽	男	2000-02-20 00:00:00	01
01640404	英雄	女	1998-03-18 00:00:00	02
01640405	李平	男	1992-04-01 00:00:00	03
01640406	马浅	女	2001-03-04 00:00:00	04

10、基于 choose 数据库先创建一个查看成绩不及格（成绩小于 50 分）的选课视图 choose_view。

执行代码：

```
create view choose_view
as select * from choose where 成绩<50;
select * from choose_view;
```

结果截图：

13 23:32:25 create view choose_view as select * from choose w... 0 row(s) affected

Result Grid　Filter Rows:　Export:

学号	课程号	成绩	选课时间
01640401	2	40	2023-12-12 20:20:36
01640404	3	0	2023-12-12 20:20:36
01640405	1	0	2023-12-12 20:20:36

11、创建一个学生成绩不及格（成绩小于 60 分）的检查视图

执行代码：

```
create view check_view as select * from choose where 成绩<60;
select * from check_view;
```

结果截图：

15　23:36:36　create view check_view as select * from choose whe...　0 row(s) affected

Result Grid　Filter Rows:　Export:

学号	课程号	成绩	选课时间
01640401	1	50	2023-12-12 20:20:36
01640401	2	40	2023-12-12 20:20:36
01640404	3	0	2023-12-12 20:20:36
01640405	1	0	2023-12-12 20:20:36

12、在 choose 数据库中，为“student”表创建视图，并通过视图

执行代码：

```
create view student_view as select * from student;
select * from student_view;
```

结果截图：

17　23:42:14　create view student_view as select * from student　0 row(s) affected

Result Grid　Filter Rows:　Export:

学号	姓名	性别	出生日期	班级编号
01640401	浅浅	女	2015-05-06 00:00:00	01
01640402	猴子	男	2000-02-20 00:00:00	02
01640403	魔兽	男	2000-02-20 00:00:00	01
01640404	英雄	女	1998-03-18 00:00:00	02
01640405	李平	男	1992-04-01 00:00:00	03
01640406	马浅	女	2001-03-04 00:00:00	04

13、在 choose 数据库中，创建视图，查看计算机学院

①创建视图

执行代码：

```
create view department_view
```

```
as select * from department where 学院名称='计算机学院';
```

结果截图：

25 23:50:13 create view department_view as select * from depart... 0 row(s) affected

学院编号	学院名称
2	土木工程学院
1	机电工程学院
5	生物工程学院
4	管理工程学院
3	计算机学院

②查看计算机学院

执行代码：

```
select * from department_view;
```

结果截图：

Result Grid　Filter Rows:

学院编号	学院名称
3	计算机学院

14、在 choose 数据库中，创建视图，查看总成绩高于“猴子”同学总成绩的学生学号和姓名。

①创建总成绩视图

执行代码：

```
create view view_总成绩 as select student.学号,student.姓名,
sum(成绩) as 总成绩 from student,choose
where student.学号=choose.学号
group by student.学号;
select * from view_总成绩;
```

结果截图：

学号	姓名	总成绩
01640401	浅浅	150
01640402	猴子	70
01640403	魔兽	170
01640404	英雄	0
01640405	李平	0

②查看高于“猴子”总成绩的学生学号和姓名

执行代码：

```
select a.学号,a.姓名 from view_总成绩 a,view_总成绩 b
where a.总成绩>b.总成绩 and b.姓名='猴子';
```

结果截图：

Result Grid　Filter Rows:

	学号	姓名
▶	01640403	魔兽
	01640401	浅浅

（三）简述视图与数据表的差异

- 表要占用磁盘空间，视图不需要
- 视图不能添加索引
- 视图是一张虚拟表，不直接储存数据，但使用视图可以简化查询
- 视图可以提高安全性
- 可以通过视图插入数据，但是只能基于一个基础表进行插入，不能跨表更新数据

（四）总结与归纳

1.创建视图

```
CREATE VIEW 视图名(列 1，列 2...)
 AS SELECT (列 1，列 2...)
 FROM ...;
```

2.修改视图

```
CREATE OR REPLACE VIEW 视图名 AS SELECT [...] FROM [...];
```

3.查看视图

①查看数据库中有哪些视图

```
SHOW TABLES;
```

②查看视图详情

```
DESC 视图名;
或者
SHOW FIELDS FROM 视图名;
```

4.插入数据

示例：

```
INSERT INTO v_order(pid,pname,price)
VALUES('p010','柴油','34');
```

5.修改视图

方式 1：

```
ALTER VIEW 视图名称 AS 查询语句
```

方式 2 示例：

```
CREATE OR REPLACE VIEW  v_student_score
AS
SELECT t1.id,t1.name,t2.c_name,t2.grade
FROM student t1 left join score t2
on t1.id = t2.stu_id
```

6.删除视图

```
DROP VIEW IF EXISTS 视图名称 1,视图名称 2,视图名称 3,...;
```

7.修改视图的数据

```
UPDATE 视图名 SET 列名='设置值' WHERE 列名='条件';
```

8.删除视图中某一数据

```
DELETE FROM 视图名 WHERE 列名='条件';
```

实验六 MySQL 函数

实验目的

- 掌握 MySQL 编程基础知识
- 了解不同运算符的优先级
- 掌握自定义函数
- 理解流程控制语句

二、实验内容

1. 创建自定义函数，根据学号查找学生所在的班级。

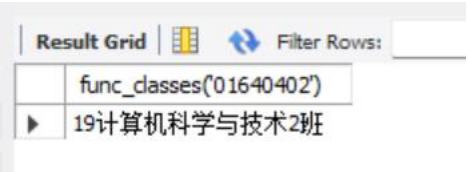
执行代码：

```
delimiter $$
create function func_classes(a char(20))
returns char(20)
reads sql data
begin
declare s char(20);
select 班级名称 into s from classes where 班级编号 in(select 班级编号 from student where 学号=a);
return s;
end;$$
```

调用函数：

```
select func_classes('01640402');
```

结果截图：



Result Grid　Filter Rows:

	func_classes('01640402')
▶	19计算机科学与技术2班

2. 创建自定义函数 cn_fn，根据班级名称统计该班级的学生总人数。调用 cn_fn 函数，返回“19 信息管理 1 班”的学生人数。

执行代码：

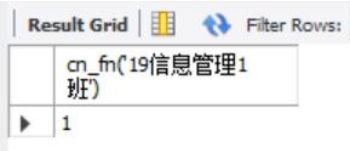
```
delimiter $$
create function cn_fn(a char(20))
returns int reads sql data
```

```
begin
declare s int;
select count(学号) into s from classes
join student on classes.班级编号 = student.班级编号
where 班级名称=a;
return s;
end;$$
```

调用代码：

```
select cn_fn('19 信息管理 1 班');
```

结果截图：



	cn_fn('19信息管理1班')
▶	1

3. 创建自定义函数，求两个数的最大数。

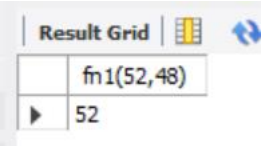
执行代码：

```
create function fn1(a int,b int)
returns int
no sql
begin
if a>b then return a;
else return b;
end if;
end;
```

调用函数：

```
select fn1(52,48);
```

结果截图：



	fn1(52,48)
▶	52

4. 自定义无参数函数，根据系统日期返回礼貌用语，分别用 if、case 条件语句。

执行代码：

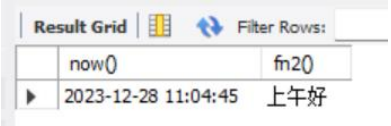
```
create function fn2()
returns char(10)
no sql
begin
declare m char(10);
```

```
if(hour(now())>6 && hour(now())<8)
then set m='早上好';
elseif(hour(now())>=8 && hour(now())<12)
then set m='上午好';
elseif(hour(now())<14 && hour(now())>=12)
then set m='中午好';
elseif(hour(now())<18)
then set m='下午好';
else set m='晚上好';
end if;
return m;
end;
```

调用函数：

```
select now(),fn2();
```

结果截图：



Result Grid　Filter Rows:

	now()	fn2()
▶	2023-12-28 11:04:45	上午好

5. 分别使用 while/repeat/循 leave 环语句创建函数，返回 1~n(n>1)的整数和。

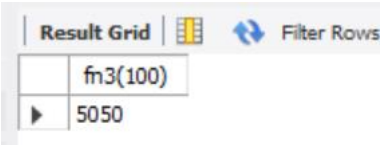
执行代码：

```
delimiter $$
create function fn3(n int)
returns int
no sql
begin
declare sum int default 0;
declare start int default 0;
repeat
set start=start+1;
set sum=sum+start;
until start=n
end repeat;
return sum;
end;$$
```

调用函数：

```
select fn3(100);$$
```

结果截图：



Result Grid　Filter Rows

	fn3(100)
▶	5050

6. 创建函数，使该函数返回 1~n（n>1）能被 9 整除的数之和。

执行代码：

```
delimiter $$
create function fn4(n int)
returns int
no sql
begin
declare sum int default 0;
declare i int default 0;
while i<n do
set i=i+1;
set sum=sum+i;
end while;
return sum;
end;$$
```

调用函数：

```
select fn4(10),fn4(50);$$
```

结果截图：

Result Grid　Filter R

	fn4(10)	fn4(50)
▶	55	1275

三、思考总结

通过练习，强化对条件控制语句与循环语句语法的理解以及语句的应用。简介主要的条件控制语句与循环语句各有哪些，它们的功能有何不同。

（1）if 语句

用来进行条件判断，根据是否满足条件（可包含多个条件），来执行不用的语句，是流程控制中最常见的判断语句

```
IF search_condition THEN statement_list
    [ELSEIF search_condition THEN statement_list]...
    [ELSE statement_list]
END IF
```

（2）case 语句

用来进行条件判断，提供了多个条件进行选择，可以实现比 if 语句更复杂的条件判断

```
CASE case_value
    WHEN when_value THEN statement_list
    [WHEN when_value THEN statement_list]...
    [ELSE statement_list]
END CASE
```

• case_value 参数表示条件判断的变量，决定了哪一个 WHEN 子句会被执行；

• when_value 参数表示变量的取值，如果某个 when_value 表达式与 case_value 变量的值相同，则执行对应的 THEN 关键字后的 statement_list 中的语句；

```
CASE
    WHEN search_condition THEN statement_list
    [WHEN search_condition THEN statement_list] ...
    [ELSE statement_list]
END CASE
```

• search_condition 参数表示条件判断语句；statement_list 参数表示不同条件的执行语句

（3）loop 语句

可以使某些特定的语句重复执行，与 if、case 相比，loop 只实现了一个简单的循环，并不进行条件判断

```
[begin_label:]LOOP
    statement_list
END LOOP [end_label]
```

• begin_label 参数和 end_label 参数分别表示循环开始和结束的标志，这两

个标志必须相同，而且都可以省略；statement_list 参数表示需要循环执行的语句

（4）leave 语句

主要用于跳出循环控制

LEAVE label

• label 参数表示循环的标志，LEAVE 语句必须跟在循环标志前面

（5）iterate 语句

Iterate 是“再次循环”的意思，用来跳出本次循环，直接进入下一次循环；iterate 语句必须跟在循环标志前面

ITERATE label

• label 参数表示循环的标志，ITERATE 语句必须跟在循环标志前面

#LEAVE 语句是跳出整个循环，然后执行循环后面的程序。而 ITERATE 语句是跳出本次循环，然后进入下一次循环。

（6）repeat 语句

用来有条件控制的循环语句，每次语句执行完毕后，会对条件表达式进行判断，如果表达式返回值未 true，则循环结束，否则重复执行循环中的语句

```
[begin_label:] REPEAT
       statement_list
       UNTIL search_condition
END REPEAT [end_label]
```

• begin_label 为 REPEAT 语句的标注名称，该参数可以省略；

• REPEAT 语句内的语句被重复，直至 search_condition 返回值为 TRUE

（7）while 语句

也是条件控制的循环语句。While 语句和 repeat 语句不同的是：while 语句是

当满足条件时，执行循环的语句，否则退出循环。

```
[begin_label:] WHILE search_condition DO
       statement list
END WHILE [end label]
```

复杂语句：

1. BEGIN…END：BEGIN…END 中间包含了多个语句，每个语句都以（**;**）号为结束符。
2. DECLARE：DECLARE 用来声明变量，使用的位置在于 **BEGIN…END** 语句中间，而且需要在其他语句使用之前进行变量的声明。
3. SET：赋值语句，用于对变量进行赋值。
4. SELECT… INTO：把从数据表中查询的结果存放到变量中，也就是为变量赋值。

设置新的结束标志：

DELIMITER 新的结束标记

实验七 存储过程与触发器

1 实验目的

1） 掌握存储过程和触发器的基本概念和功能。

2） 掌握利用 MySQL 语句创建、维护存储过程和触发器的方法。

3） 掌握利用 MySQL Workbench 创建、修改存储过程的方法。

2 实验内容及步骤

1）创建一个名为 s1_proc 的存储过程，查看学生选课数据库中学生的总人数。

执行代码：

```
delimiter $$
create procedure c_proc(in sno char(20))
reads sql data
begin
select 课程名称 from course
where 课程号 in(
select 课程号 from choose where 学号=sno);
end;$$
```

调用存储过程：

```
call c_proc('01640401');$$
```

结果截图：

61 13:27:19 create procedure s1_proc() reads sql data begin sele... 0 row(s) affected 0.016 sec

Result Grid | Filter Rows

	count(*)
▶	6

2） 创建名为 c_proc 的存储过程，根据学号查询该学生选修了哪些

课程。并调用该存储过程。

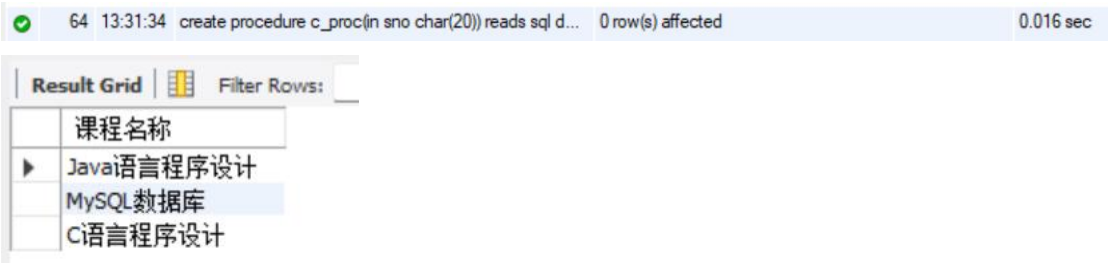
执行代码：

```
delimiter $$
create procedure c_proc(in sno char(20))
reads sql data
begin
select 课程名称 from course
where 课程号 in(
select 课程号 from choose where 学号=sno);
end;$$
```

调用储存过程：

```
call c_proc('01640401');$$
```

结果截图：



3）创建名为 c1_proc 的存储过程，根据学生的学号查询该学生选修了几门课程，s_no 是 IN 输入参数，number 是 OUT 输出参数。并调用该存储过程。

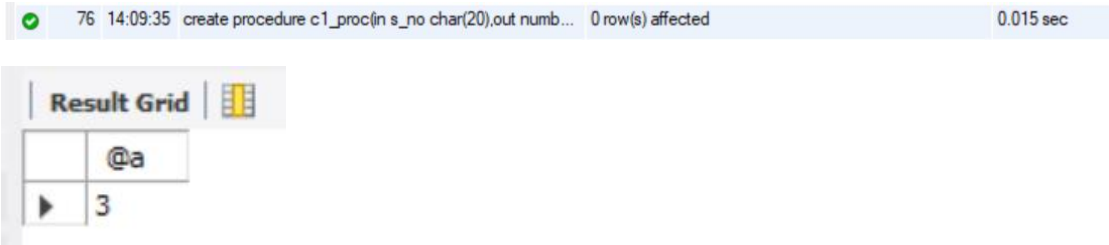
执行代码：

```
delimiter $$
create procedure c1_proc(in s_no char(20),out number int)
reads sql data
begin
select count(*) into number
from choose where 学号=s_no;
end;$$
```

调用存储过程：

```
call c1_proc('01640401',@a);$$
select @a;$$
```

结果截图：



4）创建名为 s2_proc 的存储过程，根据图书编号查询图书名称。定义 bh 为 IN 输入参数，sm 为 OUT 输出参数。

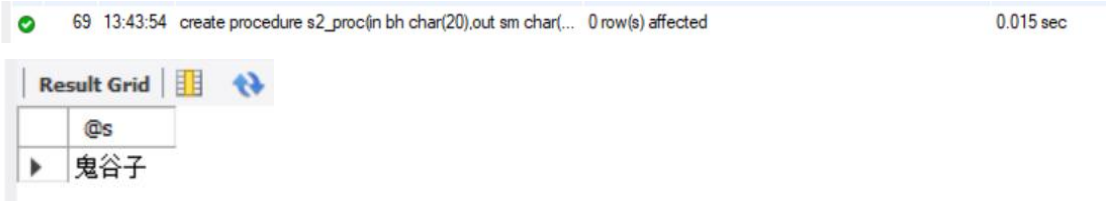
执行代码：

```
delimiter $$
create procedure s2_proc(in bh char(20),out sm char(20))
reads sql data
begin
select 图书名称 into sm from book where 图书编号=bh;
end;$$
```

调用存储过程：

```
call s2_proc('a2018002',@s);$$
select @s;$$
```

结果截图：



5）创建名为 s3_proc 的存储过程，根据图书编号查询图书名称。要求使用 INOUT 参数。调用该存储过程，输入图书编号，返回图书名称。

执行代码：

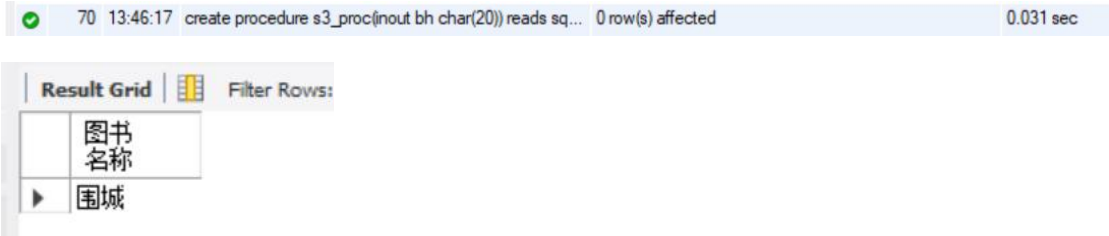
```
delimiter $$
create procedure s3_proc(inout bh char(20))
```

```
reads sql data
begin
select 图书名称 from book where 图书编号=bh;
end;$$
```

调用存储过程：

```
set @b = 'a2018001';$$
call s3_proc(@b);$$
select @b;$$
```

结果截图：

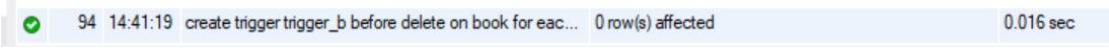


6）基于图书销售管理数据库创建删除触发器，由于《人性的弱点》这本书滞销导致下架，删除本书的信息，同时删除该书在线销售的所有信息。触发器被触发之前先打开 onsale 表查看数据，执行数据删除后对比数据。

执行代码：

```
delimiter $$
create trigger trigger_b before delete on book
for each row
begin
delete from onsale where 图书编号=old.图书编号;
end;$$
```

结果截图：



删除命令：

```
delete from book where 图书名称='人性的弱点';$$
select * from onsale;
```

```
3.  select * from book;
```

结果截图：

99 15:30:40 delete from book where 图书名称='人性的弱点'; 1 row(s) affected 0.016 sec

前后比对：

Result Grid | Filter Rows: | Edit: | Exp

图书编号	图书名称	作者	单价	出版商
a2018001	围城	钱钟书	18.00	晨光出版社
a2018002	鬼谷子	奥里森·马丁	42.00	北方文艺出版社
a2018003	人性的弱点	戴尔·卡耐基	15.00	人民出版社
a2018004	时间简史	史蒂·芬霍金	50.00	科学技术出版社
a2018005	方与圆	文德	6.80	海天出版社
a2018006	朗读者	董卿	10.50	中国人民大学出版社
a2018007	巴黎圣母院	维克多·雨果	28.00	人民文学出版社
a2018008	活着	余华	10.50	人民曲电出版社
NULL	NULL	NULL	NULL	NULL

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

图书编号	图书名称	作者	单价	出版商
a2018001	围城	钱钟书	18.00	晨光出版社
a2018002	鬼谷子	奥里森·马丁	42.00	北方文艺出版社
a2018004	时间简史	史蒂·芬霍金	50.00	科学技术出版社
a2018005	方与圆	文德	6.80	海天出版社
a2018006	朗读者	董卿	10.50	中国人民大学出版社
a2018007	巴黎圣母院	维克多·雨果	28.00	人民文学出版社
a2018008	活着	余华	10.50	人民曲电出版社

Result Grid | Filter Rows:

书店id	图书编号	上架数量
1	a2018001	425
1	a2018004	360
1	a2018005	125
1	a2018007	285
2	a2018003	389
2	a2018008	500
3	a2018001	444
3	a2018006	310
3	a2018005	428
4	a2018004	205
4	a2018006	466
4	a2018007	400
5	a2018001	405
5	a2018002	410
5	a2018003	330
5	a2018005	445
5	a2018006	421
5	a2018008	460

Result Grid | Filter Rows:

书店id	图书编号	上架数量
1	a2018001	425
1	a2018004	360
1	a2018005	125
1	a2018007	285
2	a2018008	500
3	a2018001	444
3	a2018006	310
3	a2018005	428
4	a2018004	205
4	a2018006	466
4	a2018007	400
5	a2018001	405
5	a2018002	410
5	a2018005	445
5	a2018006	421
5	a2018008	460

7）创建一张只有 a,b 两个字段的表 aa，创建插入触发器检查要添加的 a、b 的值，如果 a 的值小于 10，则允许数据添加。

执行代码：

```
1.  create table aa(a int,b int);$$
2.  delimiter $$
3.  create trigger a1 before insert on aa for each row
4.  begin
5.  if new.a<10 then set new.a=new.a;
6.  #else insert mytable values(0);
7.  end if;
8.  end;$$
```

结果截图：

124	15:42:23	create trigger a1 before insert on aa for each row beg...	0 row(s) affected	0.016 sec

输入数据：

```
insert into aa values(8,72);$$
select * from aa;$$
```

结果截图：

Result Grid

	a	b
▶	8	72

3 总结

1） 存储过程输出参数和输入参数在定义、调用时的区别。

①存储过程语法：

```
CREATE PROCEDURE 存储过程名(IN|OUT|INOUT 参数名 参数类型,...)
[characteristics ...]
BEGIN
    存储过程体

END
```

②参数类型区别

IN	输入参数:表示该参数的值必须在调用存储过程时指定赋值，在存储过程中修改该参数的值不能被返回，为默认值
OUT	输出参数:该值可在存储过程内部被改变，并可返回
INOUT	输入输出参数:调用时必须指定具体值，并且可在存储过程体重被改变和在调用完毕后可被返回

③函数调用：

• 用户变量：用户变量名一般以@开头，滥用用户变量会导致程序难以理解及管理

```
CALL 存储过程名（实例参数）;
格式：
1.调用 in 模式的参数
CALL 存储过程名（‘值’）;
2.调用 out 模式的参数
CALL 存储过程名（‘值’,@name）;
SELECT @name;
3.调用 inout 模式的参数
SRT @name=‘值’;
CALL 存储过程名（@name）;
SELECT @name;
```

2） 存储过程与存储函数的区别

·存储过程（PROCEDURE）用于完成一次完整的业务处理，没有返回值，但可通过传出参数将多个值传给调 用环境
·存储函数（FUNCTION）用于完成一次特定的计算，具有一个返回值

3） 介绍触发器

①什么是触发器

触发器是由**事件来触发**某个操作，这些事件包括 INSERT、UPDATE、DELETE 事件。所谓事件就是指用户的动作或者触发某项行为。如果定义了触发程序，当数据库执行这些语句时候，就相当于事件发生了，就会**自动**激发触发器执行相应的操作。

②创建触发器语法：

```
create trigger 触发器名 触发时机 触发事件 on 表 for each row
begin
    -- 触发器业务代码
end;
```

③触发器操作：

- 触发时间：insert、update、delete
- 触发时机：before（触发事件之前）、after（触发事件之后）
- for each row：对于其中的每个数据行，只要符合触发条件，均激活一次，也就是行级触发器；若不写，则只是对一个表设置触发器

（oracle 触发器中分行级触发器和语句级触发器，可不写 for each row，无论影响多少行都只执行一次。但 mysql 不支持语句触发器,所以必须写 for each row）

4） 触发器 insert、delete、update 操作和临时表 inserted、deleted 的关系。

当对某张表建立触发器后，分 3 种情况讨论

1.插入操作（Insert）

Inserted 表有数据，Deleted 表无数据

2.删除操作（Delete）

Inserted 表无数据，Deleted 表有数据

3.更新操作（Update）

Inserted 表有数据（新数据），Deleted 表有数据（旧数据）

• **Deleted 表**用于存储 DELETE 和 UPDATE 语句所影响的行的复本。在执行 DELETE 或 UPDATE 语句时，行从触发器表中删除，并传输到 deleted 表中。Deleted 表和触发器表通常没有相同的行。
• **Inserted 表**用于存储 INSERT 和 UPDATE 语句所影响的行的副本。在一个插入或更新事务处理中，新建行被同时添加到 inserted 表 和触发器表中。Inserted 表中的行是触发器表中新行的副本。
• 更新事务类似于在删除之后执行插入；首先旧行被复制到 deleted 表中，然后新行被复制到触发器表和 inserted 表中。

5） 触发器 insert、delete、update 操作和临时表 old、new 的关

系。

行级触发器中有**两个伪变量**，用于识别值的状态：

:old 操作之前，是记录变量，使用形式：:old.字段名

:new 操作之后，是记录变量，使用形式：:new.字段名

只在 DML 触发表中字段时才有效，只能在触发器内部使用

触发器类型	NEW和OLD的使用
insert	new表示将要或者已经插入的数据，没有old值
update	old表示更新之前的数据，new表示将要或者已经更新后的数据
delete	old表示将要或者已经删除的数据，没有new值

https://blog.csdn.net/qq_41153943

new 只出现在 insert 和 update 时，表示新插入的数据行，或者要替换的新数据行
old 只出现在 update 和 delete 时，表示要删除的数据行，或者要替换的原数据行

◈ 在使用行级触发器操作的过程之中，可以在触发器内部访问正在处理中的行数据，此时可以通过两个相关的标识符：“:old.字段”和“:new.字段”实现的，而这两个标识符仅仅是在DML触发表中字段时才会有效

No.	触发语句	:old.字段	:new.字段
1	INSERT	未定义，字段内容均为NULL	INSERT操作结束后，为增加数据值
2	UPDATE	更新数据前的原始值	UPDATE操作之后，更新数据后的新值
3	DELETE	删除前的原始值	未定义，字段内容为均为NULL

CSDN @爱吃烧烤的小肇

6） 触发器的执行顺序

当一个表中有多个触发器时，通常按如下顺序执行:

before 表级触发器

before 行级触发器

更新操作

after 行级触发器

after 表级触发器